

Day 6 Practice Assignments: Object-Oriented Programming (OOP)

Objective

Apply inheritance, encapsulation, polymorphism, and decorators to structure class-based designs. Implement multiple inheritance, properties, operator overloading, and custom iterators.

Easy Assignments

Assignment 1: Smart Home Appliance Control

Scenario

You are designing a control model for a smart home thermostat. The target temperature must be guarded against invalid bounds (e.g., set too high or too low, causing damage or excessive energy usage).

Problem Description

Create a class named `SmartThermostat` that implements the following specifications:

1. **Class-level Constant Variables:**

- `MIN_TEMP = 10.0` (float)
- `MAX_TEMP = 35.0` (float)

2. **Constructor (`__init__`):**

- Accepts parameters: `appliance_name` (string) and `initial_temp` (float).
- Sets a private attribute `__appliance_name` (assigned from `appliance_name`).
- Sets a private attribute `__target_temp` (float). Call the setter property inside the constructor or perform checks to ensure that if the `initial_temp` is out of the `[MIN_TEMP, MAX_TEMP]` bounds, it defaults to `22.0`.

3. **Properties:**

- `target_temp` (read-write property):
 - **Getter:** Returns the value of `__target_temp`.
 - **Setter:** Checks if the new temperature is within the range `[MIN_TEMP, MAX_TEMP]` inclusive. If valid, updates `__target_temp`. If invalid, raises a `ValueError` with message: `"Temperature must be between 10.0 and 35.0 degrees."`
- `appliance_name` (read-only property):
 - **Getter:** Returns `__appliance_name`.
 - (No setter defined, making it read-only after creation).

Example Walkthrough

```
thermostat = SmartThermostat("Living Room AC", 24.0)
print(thermostat.appliance_name)  # Output: Living Room AC
print(thermostat.target_temp)     # Output: 24.0
```

```

thermostat.target_temp = 28.0      # Updates successfully
print(thermostat.target_temp)     # Output: 28.0

try:
    thermostat.target_temp = 5.0   # Out of range!
except ValueError as e:
    print(e) # Output: Temperature must be between 10.0 and 35.0 degrees.

```

Assignment 2: Vehicle Fleet Management

Scenario

A delivery company manages different vehicle classes. They want to calculate travel range based on fuel capacity, and adjust range for trucks depending on cargo load.

Problem Description

- Create a base class named **Vehicle** with:
 - Constructor (`__init__`)**: Accepts **make** (string), **model** (string), and **fuel_capacity** (float, in liters).
 - Method `calculate_range(fuel_efficiency)`**: Calculates and returns the vehicle's range (in kilometers) by multiplying the **fuel_capacity** by the **fuel_efficiency** (km per liter).
 - Method `get_description()`**: Returns a formatted string: "Vehicle: <make> <model>".
- Create a subclass named **DeliveryTruck** that inherits from **Vehicle**:
 - Constructor (`__init__`)**: Accepts **make** (string), **model** (string), **fuel_capacity** (float), and **cargo_load** (float, in metric tons). Uses **`super().__init__()`** to initialize base vehicle parameters.
 - Method `calculate_range(fuel_efficiency)`**: Overrides the base method. Heavy loads reduce efficiency. Reduce the range calculation by **10% for every metric ton** of **cargo_load** currently carried.

$$\text{Adjusted Range} = \text{Base Range} \times (1.0 - 0.1 \times \text{cargo_load})$$

- Method `get_description()`**: Overrides the base method. Returns a formatted string: "Truck: <make> <model> carrying <cargo_load> tons".

Example Walkthrough

```

truck = DeliveryTruck("Volvo", "FH16", 300.0, cargo_load=2.0)

# Base range calculations without load adjustment would be 300 * 5 = 1500 km.
# 2.0 tons load reduces range by 20% (10% * 2) -> 1500 * 0.8 = 1200 km.
print(truck.calculate_range(5.0)) # Output: 1200.0

```

```
print(truck.get_description())    # Output: Truck: Volvo FH16 carrying 2.0
tons
```

Medium Assignments

Assignment 3: E-Commerce Currency Converter

Scenario

An online store represents transaction totals as structured objects containing currency labels. To prevent logic errors, the system must not add different currencies directly and should print descriptive labels.

Problem Description

Create a class named `PriceAmount` with the following requirements:

1. **Constructor (`__init__`):** Accepts `value` (float) and `currency` (string). Standardize the `currency` string value by converting it to uppercase.
2. **Dunder Methods for String Representation:**
 - `__str__`: Returns a string formatted as "`<currency> <value>`" with the value rounded to **2 decimal places** (e.g., "`USD 19.99`").
 - `__repr__`: Returns a detailed programmer representation: "`PriceAmount(value=<value>, currency='<currency>')`" (value rounded to 2 decimal places).
3. **Operator Overloading:**
 - `__add__(self, other)`:
 - Check if `other` is an instance of `PriceAmount` and has the **same** currency value.
 - If the currency values do not match, raise a `ValueError` with the message: "`Cannot add price amounts with different currencies: '<currency1>' and '<currency2>'`".
 - If valid, return a **new** `PriceAmount` instance with the summed value and the same currency.
 - `__eq__(self, other)`:
 - Returns `True` if `other` is an instance of `PriceAmount`, has the same currency, and the values are identical. Otherwise, returns `False`.

Example Walkthrough

```
p1 = PriceAmount(19.99, "usd")
p2 = PriceAmount(10.01, "USD")
p3 = PriceAmount(15.00, "EUR")

print(str(p1))      # Output: USD 19.99
print(repr(p1))     # Output: PriceAmount(value=19.99, currency='USD')

total = p1 + p2
print(str(total))   # Output: USD 30.00
```

```
print(p1 == PriceAmount(19.99, "USD")) # Output: True

try:
    bad_addition = p1 + p3
except ValueError as e:
    print(e) # Output: Cannot add price amounts with different currencies:
    'USD' and 'EUR'.
```

Assignment 4: Hospital Patient Register

Scenario

A hospital patient ledger automatically tracks patient counts and assigns sequentially numbered keys. It also validates input dates to prevent registration crashes.

Problem Description

Create a class named `Patient` that satisfies the following:

1. Class-level Variables:

- `_patient_counter` (integer, initialized to 0): Tracks the total count of patient instances created.

2. Static Method `validate_dob_format(dob_str)`:

- Uses a Regular Expression pattern to check if the date of birth matches the format "YYYY-MM-DD" exactly (4 digits, a hyphen, 2 digits, a hyphen, 2 digits).
- Returns `True` if correct, and `False` otherwise.

3. Constructor (`__init__`):

- Accepts parameters: `name` (string) and `dob` (string, representation of date of birth).
- First, calls `Patient.validate_dob_format(dob)`. If it returns `False`, raise a `ValueError` with the message: "Invalid date of birth format: '<dob>'. Expected YYYY-MM-DD."
- If validation passes, increments the class variable `_patient_counter` by 1.
- Assigns a unique `patient_id` as a string: "PAT-" followed by the value of `1000 + _patient_counter` (e.g., "PAT-1001", "PAT-1002").
- Stores `name` and `dob` as instance variables.

4. Class Method `get_total_patients()`:

- Returns the value of `_patient_counter`.

Example Walkthrough

```
# 1. Valid Registration
p1 = Patient("Arham Khan", "1999-05-15")
print(p1.patient_id) # Output: PAT-1001

# 2. Invalid DOB registration (throws ValueError)
try:
    p2 = Patient("Lisa", "12/08/1998")
```

```
except ValueError as e:
    print(e) # Output: Invalid date of birth format: '12/08/1998'.
    Expected YYYY-MM-DD.

print(Patient.get_total_patients()) # Output: 1
```

Difficult Assignments

Assignment 5: Multi-Channel Notification System (Multiple Inheritance & MRO)

Scenario

An automated incident response engine sends server health alert broadcasts. Depending on incident severity, it sends notifications via Email, SMS, or both using cooperative multiple inheritance.

Problem Description

Implement a cooperative multiple inheritance structure using the following class designs:

1. Base Class **Notifier**:

- Constructor (`__init__`): Accepts `sender_id` (string).
- Method `send(message)`: Returns a list containing the log: `["[Notifier <sender_id>] general broadcast: <message>"]`.

2. Subclass **EmailNotifier** (inherits from **Notifier**):

- Constructor (`__init__`): Accepts `email_server` (string) along with any other keyword parameters. It must forward parameters to the next class in the hierarchy using `super().__init__()` or direct calls.
- Method `send(message)`: Calls `super().send(message)` to get the log list, prepends the string `"[Email via <email_server>] sending: <message>"` to the list, and returns it.

3. Subclass **SMSNotifier** (inherits from **Notifier**):

- Constructor (`__init__`): Accepts `sms_gateway` (string) along with any other keyword parameters. It must forward parameters to the next class in the MRO.
- Method `send(message)`: Calls `super().send(message)` to get the log list, prepends the string `"[SMS via <sms_gateway>] sending: <message>"` to the list, and returns it.

4. Subclass **HybridAlertChannel** (inherits from BOTH **EmailNotifier** and **SMSNotifier** in that order):

- Constructor (`__init__`): Accepts `sender_id` (string), `email_server` (string), and `sms_gateway` (string). Passes all values cooperatively through `super().__init__()`.
- Method `send(message)`: Calls `super().send(message)` to get the consolidated log list. Prepends `"[HYBRID ALERT] Initiating dual channels..."` to the list and returns it.

5. Requirements:

- The hierarchy must support **cooperative initialization and cooperative method dispatch**. Calling `super().__init__()` or `super().send()` must pass details down the entire MRO path without skipping parent classes or duplicating calls.
- Print the Method Resolution Order (`._mro__` or `.mro()`) of `HybridAlertChannel` to verify the lookup path.

Example Walkthrough

```
alert = HybridAlertChannel(sender_id="SYS-ADMIN",
email_server="smtp.cdac.in", sms_gateway="gw.acts.com")
logs = alert.send("Disk space 95%")

for log in logs:
    print(log)
```

Expected Console Output Logs:

```
[HYBRID ALERT] Initiating dual channels...
[Email via smtp.cdac.in] sending: Disk space 95%
[SMS via gw.acts.com] sending: Disk space 95%
[Notifier SYS-ADMIN] general broadcast: Disk space 95%
```

Assignment 6: Custom Database Record Simulator

Scenario

You are developing a custom result set class to represent database queries. To make it behave like a native Python list, the object must support indexes, search lookups by name strings, iteration using custom iterator classes, and return database size.

Problem Description

1. Create a class named **DatabaseRecord**:
 - **Constructor (`__init__`)**: Accepts **record_id** (int) and **data** (dictionary).
 - **Dunder Methods**: Implement **`__repr__`** and **`__str__`** returning **"Record(id=<record_id>, data=<data>)"**.
2. Create a custom iterator class named **ResultSetIterator**:
 - **Constructor (`__init__`)**: Accepts **records_list** (list of **DatabaseRecord** instances). Initialize an index counter to **0**.
 - **Dunder Methods**:
 - **`__iter__(self)`**: Returns **self**.
 - **`__next__(self)`**: Yields the next **DatabaseRecord** in the list. If no records remain, raises **StopIteration**.
3. Create a class named **DatabaseResultSet**:
 - **Constructor (`__init__`)**: Accepts **records_list** (list of **DatabaseRecord** objects).
 - **Dunder Methods**:
 - **`__len__(self)`**: Returns the count of records in the result set.
 - **`__iter__(self)`**: Returns a new **ResultSetIterator** object initialized with **records_list**.
 - **`__getitem__(self, key)`**:

- If **key** is an **integer**, return the **DatabaseRecord** at that index. If the index is out of bounds, let it raise a standard **IndexError**.
- If **key** is a **string**, search the database records. Return the first **DatabaseRecord** object whose **data["name"]** matches the key string.
- If the name string is not found, raise a custom exception named **RecordNotFoundError** (which you must define) with the message: **"Record with name '<name>' not found in database."**

Example Walkthrough

```
# Setup records
r1 = DatabaseRecord(101, {"name": "Alice", "role": "Admin"})
r2 = DatabaseRecord(102, {"name": "Bob", "role": "User"})

results = DatabaseResultSet([r1, r2])

# 1. Length
print(len(results)) # Output: 2

# 2. Integer Indexing
print(results[0].data["role"]) # Output: Admin

# 3. String lookup
record = results["Bob"]
print(record.record_id) # Output: 102

# 4. Iteration
for rec in results:
    print(rec.record_id)
# Output:
# 101
# 102

# 5. Missing key lookup
try:
    missing = results["Charlie"]
except RecordNotFoundError as e:
    print(e) # Output: Record with name 'Charlie' not found in database.
```